

---

## YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC2 Privacy Shield Pilot — Implementation Codebase

---

Daniel Macharia &lt;danielmacharia43@gmail.com&gt;

Tue, 14 Jul at 11:34

To: &lt;daniel@ics.ac.ke&gt;

# YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC2

## Privacy Shield Pilot — Implementation Codebase

RC2 extends the immutable RC1 camera-to-render-to-upload foundation without activating captions, music, voice-over, ecosystem objects, or general AI edit execution.

The pilot introduces:

- Person and face detection.
- Stable person-track IDs.
- Selectable blur, pixelation, and silhouette protection.
- Manual protection-mask correction.
- Speaker timeline detection.
- Witness Maximum voice anonymization.
- Creative voice presets.
- Protected rendering.
- Strict separation of original and protected media.
- Server-enforced privacy publication gates.
- Allow-list and emergency-disable controls.

## Release constraint

RC2 must remain allow-listed until physical Android and iPhone tests prove:

1. No protected person appears unprotected in the final output.
2. No original protected voice leaks into the published audio.
3. The public video document references only the protected output.
4. The original source remains private and cannot be published directly.

---

## 1. Monorepo changes

```
apps/mobile_flutter/
├── lib/features/creator_studio/privacy_shield/
│   ├── domain/
│   │   ├── privacy_mode.dart
│   │   ├── privacy_shield_project.dart
│   │   ├── visual_protection_track.dart
│   │   ├── voice_protection_track.dart
│   │   └── privacy_validation_result.dart
│   ├── data/
│   │   ├── privacy_shield_repository.dart
│   │   └── firestore_privacy_shield_repository.dart
│   ├── services/
│   │   ├── person_detection_service.dart
│   │   ├── speaker_detection_service.dart
│   │   ├── privacy_render_service.dart
│   │   └── privacy_publication_service.dart
│   ├── controllers/
│   │   ├── privacy_shield_controller.dart
│   │   └── privacy_review_controller.dart
│   └── presentation/
│       ├── privacy_shield_screen.dart
│       ├── person_review_screen.dart
│       ├── speaker_review_screen.dart
│       ├── manual_mask_editor_screen.dart
│       └── protected_preview_screen.dart
└── backend/firebase_functions/src/creator-studio/privacy-shield/
    └── privacy-shield.types.ts
```

```

├── create-privacy-scan.ts
├── submit-privacy-review.ts
├── create-protected-render.ts
├── complete-protected-render.ts
├── privacy-publication-gate.ts
├── publish-protected-video.ts
├── privacy-shield.routes.ts

backend/cloud_run_workers/privacy-renderer/
├── src/
│   ├── index.ts
│   ├── render-job.ts
│   ├── visual-mask-renderer.ts
│   ├── voice-anonymizer.ts
│   ├── ffmpeg-builder.ts
│   └── storage.ts
├── Dockerfile
├── package.json
└── tsconfig.json

firestore/
├── firestore.rules
└── firestore.indexes.json

```

---

## 2. RC2 flags and allow-list

### creator\_studio\_flags.dart

```

final class CreatorStudioFlags {
  const CreatorStudioFlags._();

  static const bool enabled = bool.fromEnvironment(
    'CREATOR_STUDIO_ENABLED',
    defaultValue: false,
  );

  static const bool privacyShieldEnabled = bool.fromEnvironment(
    'CREATOR_PRIVACY_SHIELD_ENABLED',
    defaultValue: false,
  );

  static const bool visualShieldEnabled = bool.fromEnvironment(
    'CREATOR_VISUAL_SHIELD_ENABLED',
    defaultValue: false,
  );

  static const bool voiceShieldEnabled = bool.fromEnvironment(
    'CREATOR_VOICE_SHIELD_ENABLED',
    defaultValue: false,
  );

  static const bool manualMaskCorrectionEnabled =
    bool.fromEnvironment(
      'CREATOR_MANUAL_MASK_ENABLED',
      defaultValue: false,
    );

  static const bool witnessMaximumEnabled =
    bool.fromEnvironment(
      'CREATOR_WITNESS_MAXIMUM_ENABLED',
      defaultValue: false,
    );

  static const bool creativeVoiceEnabled = bool.fromEnvironment(
    'CREATOR_CREATIVE_VOICE_ENABLED',
    defaultValue: false,
  );

  static const bool privacyAllowListRequired =
    bool.fromEnvironment(
      'CREATOR_PRIVACY_ALLOW_LIST_REQUIRED',
      defaultValue: true,
    );

  static const bool privacyPublicationGateRequired =
    bool.fromEnvironment(
      'CREATOR_PRIVACY_PUBLICATION_GATE_REQUIRED',
      defaultValue: true,
    );
}

```

```

static void validateRc2() {
  if (!privacyShieldEnabled) return;

  if (!visualShieldEnabled ||
      !manualMaskCorrectionEnabled ||
      !privacyPublicationGateRequired ||
      !privacyAllowListRequired) {
    throw StateError(
      'Privacy Shield RC2 requires visual protection, manual '
      'correction, allow-listing and server publication gates.',
    );
  }
}
}

```

## Build defines

```

--dart-define=CREATOR_PRIVACY_SHIELD_ENABLED=true
--dart-define=CREATOR_VISUAL_SHIELD_ENABLED=true
--dart-define=CREATOR_VOICE_SHIELD_ENABLED=true
--dart-define=CREATOR_MANUAL_MASK_ENABLED=true
--dart-define=CREATOR_WITNESS_MAXIMUM_ENABLED=true
--dart-define=CREATOR_CREATIVE_VOICE_ENABLED=true
--dart-define=CREATOR_PRIVACY_ALLOW_LIST_REQUIRED=true
--dart-define=CREATOR_PRIVACY_PUBLICATION_GATE_REQUIRED=true

```

---

## 3. Domain models

### privacy\_mode.dart

```

enum PrivacyMode {
  standard,
  childProtection,
  protectedWitness,
  creative,
}

```

### visual\_protection\_track.dart

```

enum VisualProtectionStyle {
  gaussianBlur,
  pixelation,
  solidSilhouette,
  mosaic,
}

final class ProtectionRect {
  const ProtectionRect({
    required this.left,
    required this.top,
    required this.width,
    required this.height,
  });

  final double left;
  final double top;
  final double width;
  final double height;

  Map<String, dynamic> toJson() => {
    'left': left,
    'top': top,
    'width': width,
    'height': height,
  };

  factory ProtectionRect.fromJson(Map<String, dynamic> json) {
    return ProtectionRect(
      left: (json['left'] as num).toDouble(),
      top: (json['top'] as num).toDouble(),
      width: (json['width'] as num).toDouble(),
      height: (json['height'] as num).toDouble(),
    );
  }
}

final class VisualProtectionKeyframe {
  const VisualProtectionKeyframe({
    required this.timestampMs,
    required this.rect,
    required this.confidence,
  });
}

```

```

        required this.manuallyAdjusted,
    });

    final int timestampMs;
    final ProtectionRect rect;
    final double confidence;
    final bool manuallyAdjusted;

    Map<String, dynamic> toJson() => {
        'timestampMs': timestampMs,
        'rect': rect.toJson(),
        'confidence': confidence,
        'manuallyAdjusted': manuallyAdjusted,
    };

    factory VisualProtectionKeyframe.fromJson(
        Map<String, dynamic> json,
    ) {
        return VisualProtectionKeyframe(
            timestampMs: json['timestampMs'] as int,
            rect: ProtectionRect.fromJson(
                Map<String, dynamic>.fromJson(json['rect'] as Map),
            ),
            confidence:
                (json['confidence'] as num).toDouble(),
            manuallyAdjusted:
                json['manuallyAdjusted'] as bool ?? false,
        );
    }
}

final class VisualProtectionTrack {
    const VisualProtectionTrack({
        required this.id,
        required this.personTrackId,
        required this.startMs,
        required this.endMs,
        required this.style,
        required this.strength,
        required this.keyframes,
        required this.enabled,
        required this.reviewed,
        required this.lowConfidenceResolved,
        this.reason,
    });

    final String id;
    final String personTrackId;
    final int startMs;
    final int endMs;
    final VisualProtectionStyle style;
    final double strength;
    final List<VisualProtectionKeyframe> keyframes;
    final bool enabled;
    final bool reviewed;
    final bool lowConfidenceResolved;
    final String? reason;

    VisualProtectionTrack copyWith({
        VisualProtectionStyle? style,
        double? strength,
        List<VisualProtectionKeyframe>? keyframes,
        bool? enabled,
        bool? reviewed,
        bool? lowConfidenceResolved,
        String? reason,
    }) {
        return VisualProtectionTrack(
            id: id,
            personTrackId: personTrackId,
            startMs: startMs,
            endMs: endMs,
            style: style ?? this.style,
            strength: strength ?? this.strength,
            keyframes: keyframes ?? this.keyframes,
            enabled: enabled ?? this.enabled,
            reviewed: reviewed ?? this.reviewed,
            lowConfidenceResolved:
                lowConfidenceResolved ?? this.lowConfidenceResolved,
            reason: reason ?? this.reason,
        );
    }
}

Map<String, dynamic> toJson() => {
    'id': id,
    'personTrackId': personTrackId,
    'startMs': startMs,

```

```

        'endMs': endMs,
        'style': style.name,
        'strength': strength,
        'keyframes':
            keyframes.map((item) => item.toJson()).toList(),
        'enabled': enabled,
        'reviewed': reviewed,
        'lowConfidenceResolved': lowConfidenceResolved,
        'reason': reason,
    };
}

```

## voice\_protection\_track.dart

```

import 'privacy_mode.dart';

enum VoiceProtectionPreset {
    witnessMaximum,
    witnessNatural,
    childProtection,
    anonymousInterview,
    robot,
    deepVoice,
    cartoon,
    alien,
    mystery,
}

final class SpeakerSegment {
    const SpeakerSegment({
        required this.startMs,
        required this.endMs,
        required this.confidence,
    });

    final int startMs;
    final int endMs;
    final double confidence;

    Map<String, dynamic> toJson() => {
        'startMs': startMs,
        'endMs': endMs,
        'confidence': confidence,
    };
}

final class VoiceProtectionTrack {
    const VoiceProtectionTrack({
        required this.id,
        required this.speakerTrackId,
        required this.segments,
        required this.preset,
        required this.mode,
        required this.pitchShiftSemitones,
        required this.formantScale,
        required this.spectralWarp,
        required this.originalMix,
        required this.enabled,
        required this.reviewed,
    });

    final String id;
    final String speakerTrackId;
    final List<SpeakerSegment> segments;
    final VoiceProtectionPreset preset;
    final PrivacyMode mode;
    final double pitchShiftSemitones;
    final double formantScale;
    final double spectralWarp;

    /// Must be 0 for protected-witness publication.
    final double originalMix;

    final bool enabled;
    final bool reviewed;

    VoiceProtectionTrack copyWith({
        VoiceProtectionPreset? preset,
        PrivacyMode? mode,
        double? pitchShiftSemitones,
        double? formantScale,
        double? spectralWarp,
        double? originalMix,
        bool? enabled,
        bool? reviewed,
    }) {

```

```

return VoiceProtectionTrack(
  id: id,
  speakerTrackId: speakerTrackId,
  segments: segments,
  preset: preset ?? this.preset,
  mode: mode ?? this.mode,
  pitchShiftSemitones:
    pitchShiftSemitones ?? this.pitchShiftSemitones,
  formantScale: formantScale ?? this.formantScale,
  spectralWarp: spectralWarp ?? this.spectralWarp,
  originalMix: originalMix ?? this.originalMix,
  enabled: enabled ?? this.enabled,
  reviewed: reviewed ?? this.reviewed,
);
}

Map<String, dynamic> toJson() => {
  'id': id,
  'speakerTrackId': speakerTrackId,
  'segments':
    segments.map((item) => item.toJson()).toList(),
  'preset': preset.name,
  'mode': mode.name,
  'pitchShiftSemitones': pitchShiftSemitones,
  'formantScale': formantScale,
  'spectralWarp': spectralWarp,
  'originalMix': originalMix,
  'enabled': enabled,
  'reviewed': reviewed,
};
}

```

## privacy\_shield\_project.dart

```

import 'privacy_mode.dart';
import 'visual_protection_track.dart';
import 'voice_protection_track.dart';

enum PrivacyShieldStatus {
  notStarted,
  scanning,
  awaitingReview,
  readyToRender,
  rendering,
  validating,
  completed,
  failed,
}

final class PrivacyShieldProject {
  const PrivacyShieldProject({
    required this.projectId,
    required this.ownerId,
    required this.mode,
    required this.status,
    required this.visualTracks,
    required this.voiceTracks,
    required this.scanCompleted,
    required this.reviewCompleted,
    required this.protectedRenderCompleted,
    required this.originalPublicationBlocked,
    required this.createdAt,
    required this.updatedAt,
    this.protectedOutputAssetId,
    this.failureReason,
  });

  final String projectId;
  final String ownerId;
  final PrivacyMode mode;
  final PrivacyShieldStatus status;
  final List<VisualProtectionTrack> visualTracks;
  final List<VoiceProtectionTrack> voiceTracks;
  final bool scanCompleted;
  final bool reviewCompleted;
  final bool protectedRenderCompleted;
  final bool originalPublicationBlocked;
  final String? protectedOutputAssetId;
  final String? failureReason;
  final DateTime createdAt;
  final DateTime updatedAt;

  bool get allTracksReviewed {
    return visualTracks
      .where((track) => track.enabled)
      .every((track) =>

```

```

        track.reviewed &&
        track.lowConfidenceResolved) &&
    voiceTracks
        .where((track) => track.enabled)
        .every((track) => track.reviewed);
}

bool get readyToRender {
    return scanCompleted &&
        reviewCompleted &&
        allTracksReviewed;
}

bool get readyForPublication {
    return readyToRender &&
        protectedRenderCompleted &&
        originalPublicationBlocked &&
        protectedOutputAssetId != null;
}
}

```

---

## 4. Detection contracts

### person\_detection\_service.dart

```

import '../domain/visual_protection_track.dart';

final class DetectedPersonTrack {
    const DetectedPersonTrack({
        required this.trackId,
        required this.startMs,
        required this.endMs,
        required this.keyframes,
        required this.minimumConfidence,
    });

    final String trackId;
    final int startMs;
    final int endMs;
    final List<VisualProtectionKeyframe> keyframes;
    final double minimumConfidence;

    bool get requiresManualReview =>
        minimumConfidence < 0.80;
}

abstract interface class PersonDetectionService {
    Future<List<DetectedPersonTrack>> scan({
        required String localVideoPath,
        required int samplingIntervalMs,
    });
}

```

### speaker\_detection\_service.dart

```

import '../domain/voice_protection_track.dart';

final class DetectedSpeakerTrack {
    const DetectedSpeakerTrack({
        required this.speakerTrackId,
        required this.segments,
        required this.minimumConfidence,
    });

    final String speakerTrackId;
    final List<SpeakerSegment> segments;
    final double minimumConfidence;
}

abstract interface class SpeakerDetectionService {
    Future<List<DetectedSpeakerTrack>> scan({
        required String localVideoPath,
    });
}

```

For RC2, detection adapters may use approved on-device ML for initial person tracking and a governed backend processor for speaker diarization.

---

## 5. Privacy Shield controller

## privacy\_shield\_controller.dart

```
import 'package:flutter/foundation.dart';
import 'package:uuid/uuid.dart';

import '../domain/privacy_mode.dart';
import '../domain/privacy_shield_project.dart';
import '../domain/visual_protection_track.dart';
import '../domain/voice_protection_track.dart';
import '../services/person_detection_service.dart';
import '../services/speaker_detection_service.dart';

final class PrivacyShieldController extends ChangeNotifier {
  PrivacyShieldController({
    required PersonDetectionService personDetection,
    required SpeakerDetectionService speakerDetection,
    Uuid uuid = const Uuid(),
  }) : _personDetection = personDetection,
       _speakerDetection = speakerDetection,
       _uuid = uuid;

  final PersonDetectionService _personDetection;
  final SpeakerDetectionService _speakerDetection;
  final Uuid _uuid;

  PrivacyShieldProject? _project;
  PrivacyShieldProject? get project => _project;

  double _progress = 0;
  double get progress => _progress;

  Object? _error;
  Object? get error => _error;

  Future<void> scan({
    required String projectId,
    required String ownerId,
    required String localVideoPath,
    required PrivacyMode mode,
    required bool includeVoice,
  }) async {
    final now = DateTime.now();

    _project = PrivacyShieldProject(
      projectId: projectId,
      ownerId: ownerId,
      mode: mode,
      status: PrivacyShieldStatus.scanning,
      visualTracks: const [],
      voiceTracks: const [],
      scanCompleted: false,
      reviewCompleted: false,
      protectedRenderCompleted: false,
      originalPublicationBlocked: true,
      createdAt: now,
      updatedAt: now,
    );

    _progress = 0;
    _error = null;
    notifyListeners();

    try {
      final personFuture = _personDetection.scan(
        localVideoPath: localVideoPath,
        samplingIntervalMs:
          mode == PrivacyMode.protectedWitness ? 80 : 200,
      );

      final speakerFuture = includeVoice
        ? _speakerDetection.scan(
            localVideoPath: localVideoPath,
          )
        : Future.value(<DetectedSpeakerTrack>[]);

      final people = await personFuture;
      _progress = 0.55;
      notifyListeners();

      final speakers = await speakerFuture;

      final visualTracks = people.map((person) {
        return VisualProtectionTrack(
          id: _uuid.v4(),
          personTrackId: person.trackId,
          startMs: person.startMs,
```

```

        endMs: person.endMs,
        style: switch (mode) {
            PrivacyMode.protectedWitness =>
                VisualProtectionStyle.solidSilhouette,
            PrivacyMode.childProtection =>
                VisualProtectionStyle.pixelation,
            _ => VisualProtectionStyle.gaussianBlur,
        },
        strength:
            mode == PrivacyMode.protectedWitness ? 1 : 0.85,
        keyframes: person.keyframes,
        enabled: true,
        reviewed: false,
        lowConfidenceResolved:
            !person.requiresManualReview,
        reason: mode.name,
    );
}).toList(growable: false);

final voiceTracks = speakers.map((speaker) {
    final profile = _defaultVoiceProfile(mode);

    return VoiceProtectionTrack(
        id: _uuid.v4(),
        speakerTrackId: speaker.speakerTrackId,
        segments: speaker.segments,
        preset: profile.preset,
        mode: mode,
        pitchShiftSemitones: profile.pitch,
        formantScale: profile.formant,
        spectralWarp: profile.spectralWarp,
        originalMix: profile.originalMix,
        enabled: true,
        reviewed: false,
    );
}).toList(growable: false);

_project = PrivacyShieldProject(
    projectId: projectId,
    ownerId: ownerId,
    mode: mode,
    status: PrivacyShieldStatus.awaitingReview,
    visualTracks: visualTracks,
    voiceTracks: voiceTracks,
    scanCompleted: true,
    reviewCompleted: false,
    protectedRenderCompleted: false,
    originalPublicationBlocked: true,
    createdAt: now,
    updatedAt: DateTime.now(),
);

_progress = 1;
} catch (error) {
    _error = error;

    _project = PrivacyShieldProject(
        projectId: projectId,
        ownerId: ownerId,
        mode: mode,
        status: PrivacyShieldStatus.failed,
        visualTracks: const [],
        voiceTracks: const [],
        scanCompleted: false,
        reviewCompleted: false,
        protectedRenderCompleted: false,
        originalPublicationBlocked: true,
        failureReason: error.toString(),
        createdAt: now,
        updatedAt: DateTime.now(),
    );

    rethrow;
} finally {
    notifyListeners();
}

}

void updateVisualTrack(
    VisualProtectionTrack updated,
) {
    final current = _requireProject();

    final tracks = current.visualTracks.map((track) {
        return track.id == updated.id ? updated : track;
    }).toList(growable: false);

```

```

    _replace(visualTracks: tracks);
}

void updateVoiceTrack(
    VoiceProtectionTrack updated,
) {
    final current = _requireProject();

    final tracks = current.voiceTracks.map((track) {
        return track.id == updated.id ? updated : track;
    }).toList(growable: false);

    _replace(voiceTracks: tracks);
}

void protectEveryone() {
    final current = _requireProject();

    final tracks = current.visualTracks
        .map(
            (track) => track.copyWith(
                enabled: true,
                reviewed: true,
            ),
        )
        .toList(growable: false);

    _replace(visualTracks: tracks);
}

void completeReview() {
    final current = _requireProject();

    if (!current.allTracksReviewed) {
        throw StateError(
            'All enabled visual and voice tracks must be reviewed.',
        );
    }

    _project = PrivacyShieldProject(
        projectId: current.projectId,
        ownerId: current.ownerId,
        mode: current.mode,
        status: PrivacyShieldStatus.readyToRender,
        visualTracks: current.visualTracks,
        voiceTracks: current.voiceTracks,
        scanCompleted: current.scanCompleted,
        reviewCompleted: true,
        protectedRenderCompleted: false,
        originalPublicationBlocked:
            current.originalPublicationBlocked,
        createdAt: current.createdAt,
        updatedAt: DateTime.now(),
    );

    notifyListeners();
}

void _replace({
    List<VisualProtectionTrack>? visualTracks,
    List<VoiceProtectionTrack>? voiceTracks,
}) {
    final current = _requireProject();

    _project = PrivacyShieldProject(
        projectId: current.projectId,
        ownerId: current.ownerId,
        mode: current.mode,
        status: current.status,
        visualTracks:
            visualTracks ?? current.visualTracks,
        voiceTracks:
            voiceTracks ?? current.voiceTracks,
        scanCompleted: current.scanCompleted,
        reviewCompleted: false,
        protectedRenderCompleted:
            current.protectedRenderCompleted,
        originalPublicationBlocked:
            current.originalPublicationBlocked,
        protectedOutputAssetId:
            current.protectedOutputAssetId,
        createdAt: current.createdAt,
        updatedAt: DateTime.now(),
    );

    notifyListeners();
}

```

```

PrivacyShieldProject _requireProject() {
    final current = _project;

    if (current == null) {
        throw StateError(
            'Privacy Shield project is not initialized.',
        );
    }

    return current;
}

_VoiceProfile _defaultVoiceProfile(
    PrivacyMode mode,
) {
    return switch (mode) {
        PrivacyMode.protectedWitness =>
            const _VoiceProfile(
                preset: VoiceProtectionPreset.witnessMaximum,
                pitch: -7,
                formant: 0.68,
                spectralWarp: 0.5,
                originalMix: 0,
            ),
        PrivacyMode.childProtection =>
            const _VoiceProfile(
                preset: VoiceProtectionPreset.childProtection,
                pitch: -5,
                formant: 0.72,
                spectralWarp: 0.42,
                originalMix: 0,
            ),
        PrivacyMode.creative =>
            const _VoiceProfile(
                preset: VoiceProtectionPreset.robot,
                pitch: -1,
                formant: 0.95,
                spectralWarp: 0.55,
                originalMix: 0,
            ),
        PrivacyMode.standard =>
            const _VoiceProfile(
                preset:
                    VoiceProtectionPreset.anonymousInterview,
                pitch: -3,
                formant: 0.86,
                spectralWarp: 0.25,
                originalMix: 0,
            ),
    };
}

final class _VoiceProfile {
    const _VoiceProfile({
        required this.preset,
        required this.pitch,
        required this.formant,
        required this.spectralWarp,
        required this.originalMix,
    });

    final VoiceProtectionPreset preset;
    final double pitch;
    final double formant;
    final double spectralWarp;
    final double originalMix;
}

```

---

## 6. Manual mask correction

### Keyframe update helper

```

VisualProtectionTrack updateMaskKeyframe({
    required VisualProtectionTrack track,
    required int timestampMs,
    required ProtectionRect rect,
}) {
    final keyframes = [...track.keyframes];

    final existingIndex = keyframes.indexWhere(
        (item) => item.timestampMs == timestampMs,
    );
}

```

```

);

final updated = VisualProtectionKeyframe(
  timestampMs: timestampMs,
  rect: rect,
  confidence: 1,
  manuallyAdjusted: true,
);

if (existingIndex >= 0) {
  keyframes[existingIndex] = updated;
} else {
  keyframes.add(updated);
}

keyframes.sort(
  (a, b) => a.timestampMs.compareTo(b.timestampMs),
);

return track.copyWith(
  keyframes: keyframes,
  reviewed: true,
  lowConfidenceResolved: true,
);
}

```

## manual\_mask\_editor\_screen.dart

```

import 'package:flutter/material.dart';

import '../domain/visual_protection_track.dart';

class ManualMaskEditorScreen extends StatefulWidget {
  const ManualMaskEditorScreen({
    required this.track,
    required this.timestampMs,
    required this.onSave,
    super.key,
  });

  final VisualProtectionTrack track;
  final int timestampMs;
  final ValueChanged<VisualProtectionTrack> onSave;

  @override
  State<ManualMaskEditorScreen> createState() =>
    _ManualMaskEditorScreenState();
}

class _ManualMaskEditorScreenState
  extends State<ManualMaskEditorScreen> {
  late ProtectionRect _rect;

  @override
  void initState() {
    super.initState();

    final nearest = [...widget.track.keyframes]
      ..sort(
        (a, b) => (a.timestampMs - widget.timestampMs)
          .abs()
          .compareTo(
            (b.timestampMs - widget.timestampMs).abs(),
          ),
      );

    _rect = nearest.isEmpty
      ? const ProtectionRect(
        left: 0.25,
        top: 0.2,
        width: 0.5,
        height: 0.5,
      )
      : nearest.first.rect;
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Correct protection mask'),
        actions: [
          TextButton(
            onPressed: () {
              widget.onSave(
                updateMaskKeyframe(

```

```

        track: widget.track,
        timestampMs: widget.timestampMs,
        rect: _rect,
      ),
    ),
  );

  Navigator.of(context).pop();
},
child: const Text('Save'),
),
],
),
body: LayoutBuilder(
  builder: (context, constraints) {
    final left = _rect.left * constraints.maxWidth;
    final top = _rect.top * constraints.maxHeight;
    final width =
      _rect.width * constraints.maxWidth;
    final height =
      _rect.height * constraints.maxHeight;

    return Stack(
      children: [
        Container(color: Colors.black),
        Positioned(
          left: left,
          top: top,
          width: width,
          height: height,
          child: GestureDetector(
            onPanUpdate: (details) {
              setState(() {
                _rect = ProtectionRect(
                  left: (_rect.left +
                      details.delta.dx /
                      constraints.maxWidth)
                    .clamp(0, 1 - _rect.width),
                  top: (_rect.top +
                      details.delta.dy /
                      constraints.maxHeight)
                    .clamp(0, 1 - _rect.height),
                  width: _rect.width,
                  height: _rect.height,
                );
              });
            },
            child: DecoratedBox(
              decoration: BoxDecoration(
                border: Border.all(
                  color: Colors.white,
                  width: 3,
                ),
                color: Colors.black.withValues(
                  alpha: 0.45,
                ),
              ),
            ),
          ),
        ),
      ],
    );
  },
);
}
}

```

---

## 7. Protected rendering contract

### privacy\_render\_service.dart

```

import '../domain/privacy_shield_project.dart';

final class ProtectedRenderJob {
  const ProtectedRenderJob({
    required this.jobId,
    required this.status,
    required this.progress,
    this.outputAssetId,
    this.failureReason,
  });

  final String jobId;

```

```

    final String status;
    final double progress;
    final String? outputAssetId;
    final String? failureReason;
}

abstract interface class PrivacyRenderService {
    Future<String> start({
        required String sourceAssetId,
        required PrivacyShieldProject privacyProject,
    });

    Stream<ProtectedRenderJob> watch(String jobId);

    Future<void> cancel(String jobId);
}

```

---

## 8. Backend contracts

### privacy-shield.types.ts

```

export type PrivacyMode =
    | "standard"
    | "childProtection"
    | "protectedWitness"
    | "creative";

export type VisualProtectionStyle =
    | "gaussianBlur"
    | "pixelation"
    | "solidSilhouette"
    | "mosaic";

export interface ProtectionKeyframe {
    timestampMs: number;
    left: number;
    top: number;
    width: number;
    height: number;
    confidence: number;
    manuallyAdjusted: boolean;
}

export interface VisualProtectionTrack {
    id: string;
    personTrackId: string;
    startMs: number;
    endMs: number;
    style: VisualProtectionStyle;
    strength: number;
    enabled: boolean;
    reviewed: boolean;
    lowConfidenceResolved: boolean;
    keyframes: ProtectionKeyframe[];
}

export interface SpeakerSegment {
    startMs: number;
    endMs: number;
    confidence: number;
}

export interface VoiceProtectionTrack {
    id: string;
    speakerTrackId: string;
    segments: SpeakerSegment[];
    preset: string;
    mode: PrivacyMode;
    pitchShiftSemitones: number;
    formantScale: number;
    spectralWarp: number;
    originalMix: number;
    enabled: boolean;
    reviewed: boolean;
}

export interface CreateProtectedRenderInput {
    projectId: string;
    sourceAssetId: string;
    privacyMode: PrivacyMode;
    visualTracks: VisualProtectionTrack[];
    voiceTracks: VoiceProtectionTrack[];
}

```

---

## 9. Allow-list enforcement

### create-privacy-scan.ts

```
import {
  HttpsError,
  onCall,
} from "firebase-functions/v2/https";

import {
  FieldValue,
  getFirestore,
} from "firebase-admin/firestore";

const firestore = getFirestore();

export const createPrivacyScan = onCall(
  async (request) => {
    const uid = request.auth?.uid;

    if (!uid) {
      throw new HttpsError(
        "unauthenticated",
        "Authentication is required.",
      );
    }

    const allowList = await firestore
      .collection("creatorPrivacyAllowList")
      .doc(uid)
      .get();

    if (!allowList.exists ||
      allowList.data()?.enabled !== true) {
      throw new HttpsError(
        "permission-denied",
        "Privacy Shield is not enabled for this account.",
      );
    }

    const projectId = String(
      request.data?.projectId ?? "",
    );

    const sourceAssetId = String(
      request.data?.sourceAssetId ?? "",
    );

    if (!projectId || !sourceAssetId) {
      throw new HttpsError(
        "invalid-argument",
        "projectId and sourceAssetId are required.",
      );
    }

    const scanRef = firestore
      .collection("privacyShieldScans")
      .doc();

    await scanRef.set({
      id: scanRef.id,
      projectId,
      sourceAssetId,
      ownerId: uid,
      status: "queued",
      createdAt: FieldValue.serverTimestamp(),
      updatedAt: FieldValue.serverTimestamp(),
    });

    return {
      scanId: scanRef.id,
      status: "queued",
    };
  },
);
```

---

## 10. Protected render creation

### create-protected-render.ts

```

import {
  HttpsError,
  onCall,
} from "firebase-functions/v2/https";

import {
  FieldValue,
  getFirestore,
} from "firebase-admin/firestore";

import type {
  CreateProtectedRenderInput,
} from "../privacy-shield.types";

const firestore = getFirestore();

export const createProtectedRender = onCall(
  async (request) => {
    const uid = request.auth?.uid;

    if (!uid) {
      throw new HttpsError(
        "unauthenticated",
        "Authentication is required.",
      );
    }

    const input =
      request.data as CreateProtectedRenderInput;

    const project = await firestore
      .collection("creatorProjects")
      .doc(input.projectId)
      .get();

    if (!project.exists ||
      project.data()?.ownerId !== uid) {
      throw new HttpsError(
        "permission-denied",
        "The creator project is unavailable.",
      );
    }

    const enabledVisual =
      input.visualTracks.filter((item) => item.enabled);

    const enabledVoices =
      input.voiceTracks.filter((item) => item.enabled);

    const visualValid = enabledVisual.every(
      (track) =>
        track.reviewed &&
        track.lowConfidenceResolved &&
        track.keyframes.length > 0,
    );

    const voiceValid = enabledVoices.every(
      (track) =>
        track.reviewed &&
        (track.mode !== "protectedWitness" ||
          track.originalMix === 0),
    );

    if (!visualValid || !voiceValid) {
      throw new HttpsError(
        "failed-precondition",
        "Every enabled privacy track must be reviewed.",
      );
    }

    const sourceAsset = await firestore
      .collection("creatorSourceAssets")
      .doc(input.sourceAssetId)
      .get();

    if (!sourceAsset.exists ||
      sourceAsset.data()?.ownerId !== uid) {
      throw new HttpsError(
        "permission-denied",
        "Source media is unavailable.",
      );
    }

    const renderRef = firestore
      .collection("privacyRenderJobs")
      .doc();
  }
);

```

```

    await renderRef.set({
      id: renderRef.id,
      ownerId: uid,
      projectId: input.projectId,
      sourceAssetId: input.sourceAssetId,
      privacyMode: input.privacyMode,
      visualTracks: input.visualTracks,
      voiceTracks: input.voiceTracks,
      status: "queued",
      progress: 0,
      createdAt: FieldValue.serverTimestamp(),
      updatedAt: FieldValue.serverTimestamp(),
    });

    return {
      jobId: renderRef.id,
      status: "queued",
    };
  },
);

```

---

## 11. Privacy publication gate

### privacy-publication-gate.ts

```

export interface PrivacyPublicationEvidence {
  privacyRequired: boolean;
  scanCompleted: boolean;
  reviewCompleted: boolean;
  lowConfidenceResolved: boolean;
  protectedRenderCompleted: boolean;
  protectedOutputAssetId?: string;
  originalAssetPublicationBlocked: boolean;
  protectedWitnessVoiceTracks: Array<{
    enabled: boolean;
    originalMix: number;
    reviewed: boolean;
  }>;
}

export interface PrivacyPublicationDecision {
  allowed: boolean;
  blockers: string[];
}

export function evaluatePrivacyPublicationGate(
  evidence: PrivacyPublicationEvidence,
): PrivacyPublicationDecision {
  if (!evidence.privacyRequired) {
    return {
      allowed: true,
      blockers: [],
    };
  }

  const blockers: string[] = [];

  if (!evidence.scanCompleted) {
    blockers.push("Privacy scanning is incomplete.");
  }

  if (!evidence.reviewCompleted) {
    blockers.push(
      "Detected people and speakers require review.",
    );
  }

  if (!evidence.lowConfidenceResolved) {
    blockers.push(
      "Low-confidence protection segments remain unresolved.",
    );
  }

  if (!evidence.protectedRenderCompleted) {
    blockers.push(
      "Protected rendering is incomplete.",
    );
  }

  if (!evidence.protectedOutputAssetId) {
    blockers.push(
      "Protected output asset is missing.",
    );
  }
}

```

```

    }

    if (!evidence.originalAssetPublicationBlocked) {
      blockers.push(
        "Original source media is still publishable.",
      );
    }

    const unsafeWitnessVoice =
      evidence.protectedWitnessVoiceTracks.some(
        (track) =>
          track.enabled &&
          (!track.reviewed || track.originalMix !== 0),
      );

    if (unsafeWitnessVoice) {
      blockers.push(
        "Protected-witness voice transformation is incomplete.",
      );
    }

    return {
      allowed: blockers.length === 0,
      blockers,
    };
  }
}

```

---

## 12. Protected publication function

### publish-protected-video.ts

```

import {
  HttpsError,
  onCall,
} from "firebase-functions/v2/https";

import {
  FieldValue,
  getFirestore,
} from "firebase-admin/firestore";

import {
  evaluatePrivacyPublicationGate,
} from "../privacy-publication-gate";

const firestore = getFirestore();

interface PublishProtectedVideoInput {
  projectId: string;
  uploadId: string;
  privacyRenderJobId: string;
  title: string;
  caption: string;
  visibility: "public" | "followers" | "private";
}

export const publishProtectedVideo = onCall(
  async (request) => {
    const uid = request.auth?.uid;

    if (!uid) {
      throw new HttpsError(
        "unauthenticated",
        "Authentication is required.",
      );
    }

    const input =
      request.data as PublishProtectedVideoInput;

    const renderRef = firestore
      .collection("privacyRenderJobs")
      .doc(input.privacyRenderJobId);

    const render = await renderRef.get();

    if (!render.exists ||
      render.data()?.ownerId !== uid) {
      throw new HttpsError(
        "permission-denied",
        "Privacy render job is unavailable.",
      );
    }
  }
)

```

```

const data = render.data()!;

const enabledVisual =
  (data.visualTracks as Array<Record<string, unknown>>)
    .filter((track) => track.enabled === true);

const lowConfidenceResolved =
  enabledVisual.every(
    (track) =>
      track.reviewed === true &&
      track.lowConfidenceResolved === true,
  );

const protectedWitnessVoiceTracks =
  (data.voiceTracks as Array<Record<string, unknown>>)
    .filter(
      (track) =>
        track.mode === "protectedWitness",
    )
    .map((track) => ({
      enabled: track.enabled === true,
      reviewed: track.reviewed === true,
      originalMix: Number(track.originalMix ?? 1),
    }));

const decision = evaluatePrivacyPublicationGate({
  privacyRequired: true,
  scanCompleted: data.scanCompleted === true,
  reviewCompleted: data.reviewCompleted === true,
  lowConfidenceResolved,
  protectedRenderCompleted:
    data.status === "completed",
  protectedOutputAssetId:
    typeof data.outputAssetId === "string"
      ? data.outputAssetId
      : undefined,
  originalAssetPublicationBlocked:
    data.originalPublicationBlocked === true,
  protectedWitnessVoiceTracks,
});

if (!decision.allowed) {
  await firestore
    .collection("privacyPublicationAttempts")
    .add({
      ownerId: uid,
      projectId: input.projectId,
      privacyRenderJobId:
        input.privacyRenderJobId,
      allowed: false,
      blockers: decision.blockers,
      createdAt: FieldValue.serverTimestamp(),
    });

  throw new HttpsError(
    "failed-precondition",
    "Privacy publication requirements have not passed.",
    {
      blockers: decision.blockers,
    },
  );
}

const protectedAsset = await firestore
  .collection("protectedCreatorAssets")
  .doc(data.outputAssetId)
  .get();

if (!protectedAsset.exists ||
  protectedAsset.data()?.ownerId !== uid) {
  throw new HttpsError(
    "failed-precondition",
    "Protected output could not be verified.",
  );
}

const protectedData = protectedAsset.data()!;

const videoRef =
  firestore.collection("videos").doc();

await firestore.runTransaction(
  async (transaction) => {
    transaction.set(videoRef, {
      id: videoRef.id,
      ownerId: uid,
    });
  }
);

```

```

        userId: uid,
        projectId: input.projectId,
        uploadId: input.uploadId,
        privacyRenderJobId:
            input.privacyRenderJobId,
        protectedAssetId: data.outputAssetId,
        sourceAssetId: null,
        title: input.title.trim(),
        caption: input.caption.trim(),
        visibility: input.visibility,
        status: "published",
        privacyShieldApplied: true,
        privacyMode: data.privacyMode,
        videoUrl: protectedData.videoUrl,
        hlsUrl: protectedData.hlsUrl,
        previewUrl:
            protectedData.previewUrl ?? null,
        thumbnailUrl:
            protectedData.thumbnailUrl ?? null,
        createdAt: FieldValue.serverTimestamp(),
        updatedAt: FieldValue.serverTimestamp(),
    });

    transaction.update(renderRef, {
        status: "published",
        videoId: videoRef.id,
        updatedAt: FieldValue.serverTimestamp(),
    });
},
);

return {
    published: true,
    videoId: videoRef.id,
    protectedAssetId: data.outputAssetId,
};
},
);

```

The function deliberately writes:

```
sourceAssetId: null
```

to the public video document.

## 13. Storage isolation

### Storage layout

```

private-creator-source/{ownerId}/{projectId}/original.mp4
private-privacy-masks/{ownerId}/{projectId}/{jobId}/mask.mov
private-privacy-audio/{ownerId}/{projectId}/{jobId}/voice.wav
protected-creator-output/{ownerId}/{projectId}/{jobId}/protected.mp4
published-videos/{videoId}/...

```

### Storage rules

```

rules_version = '2';

service firebase.storage {
  match /b/{bucket}/o {
    match /private-creator-source/{userId}/{allPaths=**} {
      allow read, write: if request.auth != null
        && request.auth.uid == userId;
    }

    match /private-privacy-masks/{userId}/{allPaths=**} {
      allow read: if request.auth != null
        && request.auth.uid == userId;
      allow write: if false;
    }

    match /private-privacy-audio/{userId}/{allPaths=**} {
      allow read: if request.auth != null
        && request.auth.uid == userId;
      allow write: if false;
    }

    match /protected-creator-output/{userId}/{allPaths=**} {
      allow read: if request.auth != null
        && request.auth.uid == userId;
      allow write: if false;
    }
  }
}

```

```

    }

    match /published-videos/{allPaths=**} {
      allow read: if true;
      allow write: if false;
    }
  }
}

```

Only trusted backend workers may write mask, transformed-audio, protected-output, and published assets.

---

## 14. Firestore rules

```

match /creatorPrivacyAllowList/{userId} {
  allow read: if request.auth != null
    && request.auth.uid == userId;

  allow write: if false;
}

match /privacyShieldScans/{scanId} {
  allow read: if request.auth != null
    && resource.data.ownerId == request.auth.uid;

  allow create, update, delete: if false;
}

match /privacyRenderJobs/{jobId} {
  allow read: if request.auth != null
    && resource.data.ownerId == request.auth.uid;

  allow create, update, delete: if false;
}

match /protectedCreatorAssets/{assetId} {
  allow read: if request.auth != null
    && resource.data.ownerId == request.auth.uid;

  allow create, update, delete: if false;
}

match /privacyPublicationAttempts/{attemptId} {
  allow read: if request.auth != null
    && resource.data.ownerId == request.auth.uid;

  allow create, update, delete: if false;
}

match /videos/{videoId} {
  allow read: if resource.data.status == "published"
    && resource.data.visibility == "public";

  allow create, update, delete: if false;
}

```

---

## 15. Rendering worker interface

### render-job.ts

```

import type {
  CreateProtectedRenderInput,
} from "../../firebase-functions/src/creator-studio/privacy-shield/privacy-shield.types";

export interface PrivacyRenderJob
  extends CreateProtectedRenderInput {
  jobId: string;
  ownerId: string;
  sourceStoragePath: string;
  outputStoragePath: string;
}

export interface PrivacyRenderResult {
  jobId: string;
  outputStoragePath: string;
  durationMs: number;
  visualTrackCount: number;
  voiceTrackCount: number;
  sourceAudioLeakDetected: boolean;
  unprotectedFrameCount: number;
}

```

## ffmpeg-builder.ts

```
export interface RenderAssetPaths {
  inputVideo: string;
  visualMaskVideo?: string;
  transformedAudio?: string;
  outputVideo: string;
}

export function buildProtectedRenderArgs(
  paths: RenderAssetPaths,
): string[] {
  const args: string[] = [
    "-i",
    paths.inputVideo,
  ];

  if (paths.visualMaskVideo) {
    args.push("-i", paths.visualMaskVideo);
  }

  if (paths.transformedAudio) {
    args.push("-i", paths.transformedAudio);
  }

  const filterParts: string[] = [];

  if (paths.visualMaskVideo) {
    filterParts.push(
      "[0:v][1:v]overlay=0:0:format=auto[protected_video]",
    );
  }

  const videoMap = paths.visualMaskVideo
    ? "[protected_video]"
    : "0:v";

  const audioMap = paths.transformedAudio
    ? paths.visualMaskVideo
      ? "2:a"
      : "1:a"
    : "0:a?";

  if (filterParts.length > 0) {
    args.push(
      "-filter_complex",
      filterParts.join(";"),
    );
  }

  args.push(
    "-map",
    videoMap,
    "-map",
    audioMap,
    "-c:v",
    "libx264",
    "-preset",
    "medium",
    "-crf",
    "20",
    "-pix_fmt",
    "yuv420p",
    "-c:a",
    "aac",
    "-b:a",
    "160k",
    "-movflags",
    "+faststart",
    "-shortest",
    "-y",
    paths.outputVideo,
  );

  return args;
}
```

The generated visual-mask video must contain the actual blur, mosaic, or silhouette compositing. It is not sufficient to upload coordinates without applying them during rendering.

---

## 16. Voice anonymization safety rules

For protectedWitness and childProtection:

```

export function assertPrivacyVoiceProfile(
  mode: string,
  originalMix: number,
  reviewed: boolean,
): void {
  if (
    (mode === "protectedWitness" ||
     mode === "childProtection") &&
    originalMix !== 0
  ) {
    throw new Error(
      "Original voice must not remain in the privacy mix.",
    );
  }

  if (!reviewed) {
    throw new Error(
      "Voice protection track has not been reviewed.",
    );
  }
}

```

Creative voice presets must be labeled:

Entertainment effect — not certified for high-risk anonymity.

---

## 17. Privacy Shield entry screen

### privacy\_shield\_screen.dart

```

import 'package:flutter/material.dart';

import '../controllers/privacy_shield_controller.dart';
import '../domain/privacy_mode.dart';

class PrivacyShieldScreen extends StatefulWidget {
  const PrivacyShieldScreen({
    required this.controller,
    required this.projectId,
    required this.ownerId,
    required this.localVideoPath,
    required this.onContinue,
    super.key,
  });

  final PrivacyShieldController controller;
  final String projectId;
  final String ownerId;
  final String localVideoPath;
  final VoidCallback onContinue;

  @override
  State<PrivacyShieldScreen> createState() =>
    _PrivacyShieldScreenState();
}

class _PrivacyShieldScreenState
  extends State<PrivacyShieldScreen> {
  PrivacyMode _mode = PrivacyMode.standard;
  bool _includeVoice = true;

  @override
  void initState() {
    super.initState();
    widget.controller.addListener(_refresh);
  }

  void _refresh() {
    if (mounted) setState(() {});
  }

  @override
  void dispose() {
    widget.controller.removeListener(_refresh);
    super.dispose();
  }

  Future<void> _scan() async {
    await widget.controller.scan(
      projectId: widget.projectId,
      ownerId: widget.ownerId,
      localVideoPath: widget.localVideoPath,
      mode: _mode,

```

```

        includeVoice: _includeVoice,
    );
}

@override
Widget build(BuildContext context) {
    final project = widget.controller.project;

    return Scaffold(
        appBar: AppBar(
            title: const Text('YohPal Privacy Shield'),
        ),
        body: ListView(
            padding: const EdgeInsets.all(16),
            children: [
                DropdownButtonFormField<PrivacyMode>(
                    value: _mode,
                    decoration: const InputDecoration(
                        labelText: 'Protection mode',
                    ),
                    items: PrivacyMode.values.map((mode) {
                        return DropdownMenuItem(
                            value: mode,
                            child: Text(mode.name),
                        );
                    }).toList(),
                    onChanged: (value) {
                        if (value != null) {
                            setState(() => _mode = value);
                        }
                    },
                ),
                SwitchListTile(
                    title: const Text('Protect selected voices'),
                    value: _includeVoice,
                    onChanged: (value) {
                        setState(() => _includeVoice = value);
                    },
                ),
                const SizedBox(height: 16),
                FilledButton.icon(
                    onPressed:
                        widget.controller.progress > 0 &&
                        widget.controller.progress < 1
                        ? null
                        : _scan,
                    icon: const Icon(Icons.shield_outlined),
                    label: const Text('Scan video'),
                ),
                if (widget.controller.progress > 0)
                    Padding(
                        padding: const EdgeInsets.only(top: 16),
                        child: LinearProgressIndicator(
                            value: widget.controller.progress,
                        ),
                    ),
                if (project != null &&
                    project.scanCompleted) ...[
                    const SizedBox(height: 24),
                    Text(
                        '${project.visualTracks.length} people detected',
                    ),
                    Text(
                        '${project.voiceTracks.length} speakers detected',
                    ),
                    const SizedBox(height: 12),
                    OutlinedButton(
                        onPressed:
                            widget.controller.protectEveryone,
                        child: const Text('Protect everyone'),
                    ),
                    FilledButton(
                        onPressed: project.readyToRender
                            ? widget.onContinue
                            : null,
                        child: const Text(
                            'Review and render protected version',
                        ),
                    ),
                ],
            ],
        ),
    );
}

```

# 18. Required automated tests

## Privacy gate test

```
import {
  describe,
  expect,
  it,
} from "vitest";

import {
  evaluatePrivacyPublicationGate,
} from "../../src/creator-studio/privacy-shield/privacy-publication-gate";

describe("Privacy publication gate", () => {
  it("blocks missing protected output", () => {
    const result = evaluatePrivacyPublicationGate({
      privacyRequired: true,
      scanCompleted: true,
      reviewCompleted: true,
      lowConfidenceResolved: true,
      protectedRenderCompleted: true,
      originalAssetPublicationBlocked: true,
      protectedWitnessVoiceTracks: [],
    });

    expect(result.allowed).toBe(false);
    expect(result.blockers).toContain(
      "Protected output asset is missing.",
    );
  });

  it("blocks witness voice with original leakage", () => {
    const result = evaluatePrivacyPublicationGate({
      privacyRequired: true,
      scanCompleted: true,
      reviewCompleted: true,
      lowConfidenceResolved: true,
      protectedRenderCompleted: true,
      protectedOutputAssetId: "protected-1",
      originalAssetPublicationBlocked: true,
      protectedWitnessVoiceTracks: [
        {
          enabled: true,
          reviewed: true,
          originalMix: 0.2,
        },
      ],
    });

    expect(result.allowed).toBe(false);
  });

  it("allows verified protected output", () => {
    const result = evaluatePrivacyPublicationGate({
      privacyRequired: true,
      scanCompleted: true,
      reviewCompleted: true,
      lowConfidenceResolved: true,
      protectedRenderCompleted: true,
      protectedOutputAssetId: "protected-1",
      originalAssetPublicationBlocked: true,
      protectedWitnessVoiceTracks: [
        {
          enabled: true,
          reviewed: true,
          originalMix: 0,
        },
      ],
    });

    expect(result.allowed).toBe(true);
  });
});
```

## Flutter readiness test

```
void main() {
  test('privacy project cannot render before review', () {
    final project = PrivacyShieldProject(
      projectId: 'project-1',
      ownerId: 'owner-1',
      mode: PrivacyMode.protectedWitness,
```

```

        status: PrivacyShieldStatus.awaitingReview,
        visualTracks: const [],
        voiceTracks: const [],
        scanCompleted: true,
        reviewCompleted: false,
        protectedRenderCompleted: false,
        originalPublicationBlocked: true,
        createdAt: DateTime(2026),
        updatedAt: DateTime(2026),
    );

    expect(project.readyToRender, isFalse);
    expect(project.readyForPublication, isFalse);
  });
}

```

---

## 19. Mandatory physical-device test matrix

### Android and iPhone visual tests

- One stationary face.
- Rapidly moving person.
- Multiple people crossing.
- Side profile.
- Person turns their back.
- Face partially covered.
- Person exits and re-enters.
- Low-light person.
- Mirror reflection.
- Window reflection.
- Small distant person.
- Person appearing for fewer than two seconds.
- Manual correction of a missed mask.
- Protect one person while leaving another visible.
- Protect everyone.
- Solid silhouette mode.
- Pixelation mode.
- Strong blur mode.

### Voice tests

- One speaker.
- Two speakers.
- Overlapping speech.
- Whispering.
- Loud background noise.
- Protected speaker followed by unprotected speaker.
- Witness Maximum.
- Child Protection.
- Anonymous Interview.
- Robot.
- Deep Voice.
- Cartoon.
- Original audio leak inspection.
- Captured background names or addresses.
- Audio/video synchronization after transformation.

### Publication tests

- Attempt to publish original source.
  - Attempt to publish before scan.
  - Attempt to publish before review.
  - Attempt to publish unresolved low-confidence mask.
  - Attempt to publish witness voice with `originalMix > 0`.
  - Attempt to publish missing protected output.
  - Successful protected publication.
  - Verify public document contains no original-source URL.
- 

## 20. Required evidence package

```

release-evidence/
└─ creator-studio-1.1a-rc2-privacy-shield/

```

- 01-repository/
- 02-build/
- 03-allow-list/
- 04-android-visual/
- 05-ios-visual/
- 06-android-voice/
- 07-ios-voice/
- 08-manual-correction/
- 09-protected-render/
- 10-original-isolation/
- 11-publication-gate/
- 12-crashlytics/
- 13-privacy-leak-review/
- 14-defects/
- 15-governance/

Developers and QA must return:

1. Feature and release branches.
  2. Full 40-character commit.
  3. Annotated RC2 tag.
  4. Clean working-tree proof.
  5. Allow-list proof.
  6. Android visual-protection evidence.
  7. iPhone visual-protection evidence.
  8. Android voice-protection evidence.
  9. iPhone voice-protection evidence.
  10. Manual mask-correction evidence.
  11. Protected-output rendering proof.
  12. Original-source isolation proof.
  13. Server publication-blocking proof.
  14. Public video record proving protected-only URLs.
  15. Crashlytics evidence.
  16. Flutter analysis and tests.
  17. Backend build and tests.
  18. Privacy-render worker tests.
  19. Defect register.
  20. Final GO, CONDITIONAL GO, or NO-GO recommendation.
- 

## Release decision

### GO

Issue only when:

- Visual protection passes all mandatory Android and iPhone cases.
- No reviewed protected person is visible in any tested final frame.
- Witness Maximum contains no original voice in the published mix.
- Manual correction works.
- Only protected output is publicly accessible.
- Original media remains private.
- Server gates reject all bypass attempts.
- No open Critical or P0 privacy defect remains.

### CONDITIONAL GO

Allowed only for a restricted allow-listed pilot where:

- Original-source isolation and publication gates fully pass.
- No verified identity leak exists in tested protected outputs.
- Remaining issue is non-critical usability or performance.
- High-risk Protected Witness use remains disabled.
- A named owner and correction deadline are recorded.

### NO-GO

Mandatory where:

- Any protected person appears unblurred.
- A mask is lost after movement or scene re-entry.
- Original protected voice is audible.
- The original source can be published or publicly downloaded.

- A public video references an original-source asset.
- Manual correction is unavailable.
- Android or iPhone evidence is missing.
- The evidence does not match the immutable RC2 tag.

## Next Smart Move

EXECUTE: YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC2 — PRIVACY SHIELD PHYSICAL DEVICE VALIDATION

Use one immutable RC2 commit and annotated tag.

Test visual protection first on Android and iPhone, followed by voice anonymization, manual correction, protected rendering, original-source isolation, and server publication-gate bypass attempts.

The Release Review Board must issue exactly one decision:

GO  
CONDITIONAL GO  
NO-GO

Protected Witness mode must remain disabled for public users until the Board independently confirms that no protected face or person appears unprotected, no original protected voice leaks into the output, and the original source cannot be published or accessed through any public video path.